E-Commerce Management System Backend

Project Name :- E-Commerce Management System Backend

Project Type :- Internship Project

College :- Karmaveer Bhaurao Patil College of Engineering, Satara

Internship :- Symbiosis Powered by Capgemini

1. Team Information

Role	Member Name
Backend Developer	Onkar Shinde
Backend Developer	Onkar Pawar
Backend Developer	Jyotiram Kokane
Backend Developer	Abhayraj Gaikwad
Guide/Mentor	Mahajan Mungal

2. Project Overview

Description :-

The E-Commerce Management System Backend is a scalable REST API application developed using Java Spring Boot. The system manages users, companies, products, categories, orders, payments, shipping, reviews, customer support, and organizational hierarchies.

The application follows a layered architecture:

3. Technology Stack

Backend Technologies

- Java 17
- Spring Boot
- Spring Data JPA
- Hibernate ORM
- REST APIs
- Maven

Database

- MySQL

Development Tools

- IntelliJ IDEA / Eclipse
- Postman
- Git
- GitHub

Additional Libraries

- Lombok
- Jakarta Persistence API
- Spring Web
- Spring Validation

4. Project Statistics

Based on the current project structure:

Component	Count
Entities	31
Repositories	31
Service Interfaces	155
Service Implementations	155
Controllers	31
Database Tables	33
REST APIs	100+
Relationships	OneToOne, OneToMany, ManyToOne, ManyToMany

5. Layer-wise Architecture

Entity Layer

Purpose:

- Represents database tables.
- Uses JPA annotations.

Examples:

- User
- Product
- Category
- SubCategory
- Order
- Invoice
- Address
- Company

Common Annotations:

```
@Entity
@Table
@Id
@GeneratedValue
@OneToMany
@ManyToOne
```

Repository Layer

Purpose:

- Database communication.
- CRUD operations.

Example:

```
public interface ProductRepository
extends JpaRepository<Product, Long> {
}
```

Responsibilities:

- Save Data
- Fetch Data
- Update Data
- Delete Data

Service Layer

Purpose:

- Defines business operations.

Each module contains services such as:

Create Service

`addData()`

Fetch Service

`fetchById()`

Fetch All Service

`fetchAll()`

Update Service

`updateData()`

Delete Service

`deleteById()`

Service Implementation Layer

Purpose:

- Contains actual business logic.

Example:

```
@Service
public class ProductCreateServiceImpl
implements ProductCreateService
{
    @Autowired
    private ProductRepository repository;

    @Override
    public Product addData(Product product)
    {
        return repository.save(product);
    }
}
```

Controller Layer & API Testing

Put Swagger Screenshot Here

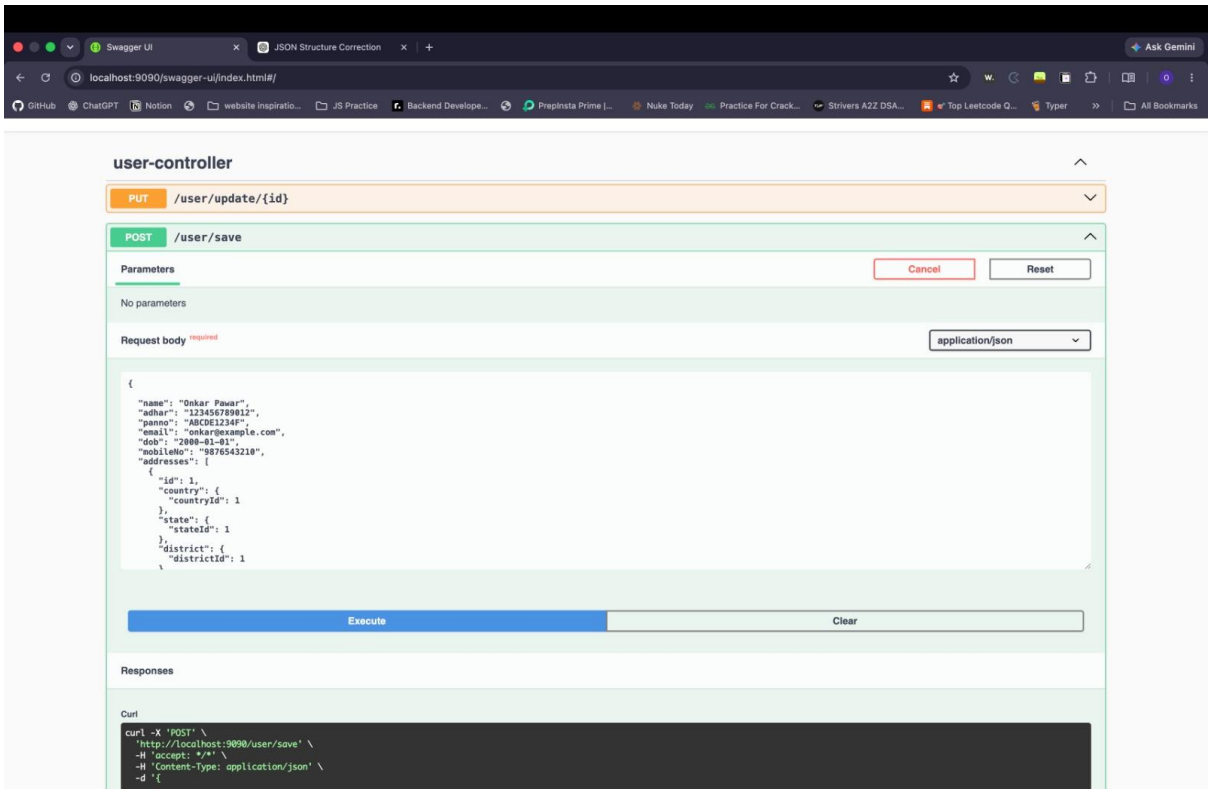
Use the **Swagger UI image** (second screenshot).

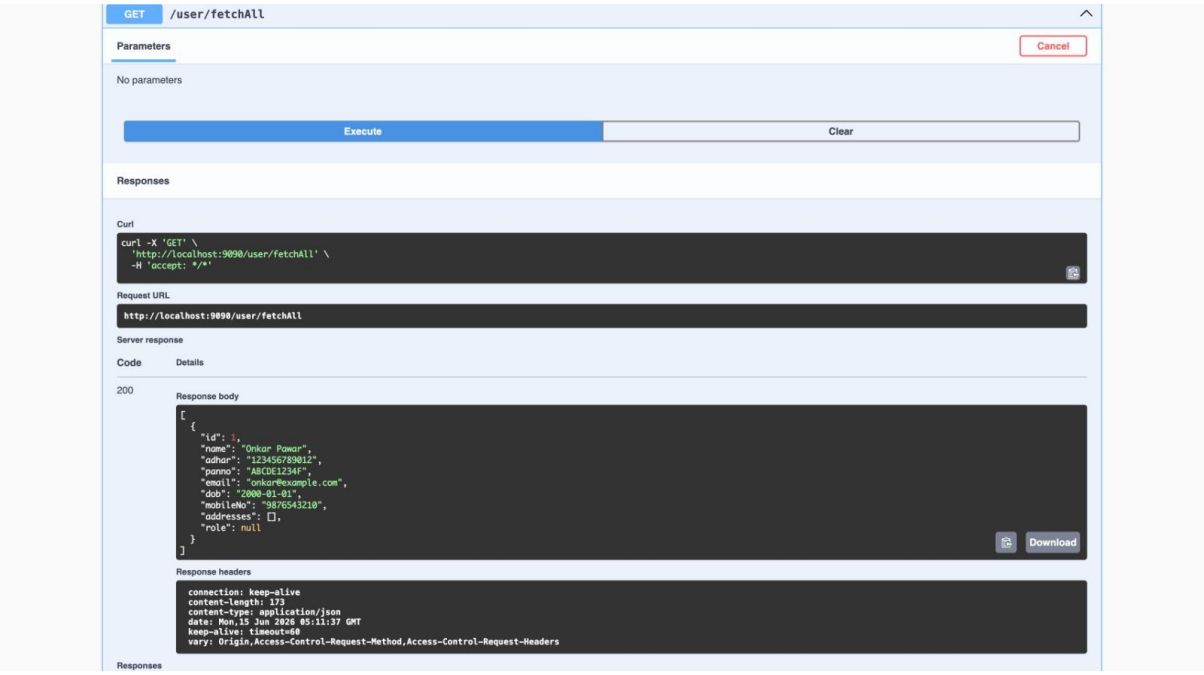
Heading

REST API Testing using Swagger UI

Points

- POST APIs
- GET APIs
- PUT APIs
- DELETE APIs
- JSON Request/Response Testing
- API Documentation





---

6. Database Modules

The project contains 33 major tables and dependency relationships.

Master Data

1. Roles
2. Country
3. Category
4. Company Type
5. Owner
6. Payment Mode

Location Hierarchy

```
Country
  └── State
        └── District
              └── Taluka
                    └── Town
```

Company Management

```
Owner
  └── Company
        ├── Manager
        ├── Department
        ├── Employee
        ├── Brand
        └── Address
```

Product Management

```
Category
  └── SubCategory
        └── Product
              └── ProductReview
```

Customer Management

```
User
  ├── Feedback
  ├── CustomerQuery
  └── Orders
```

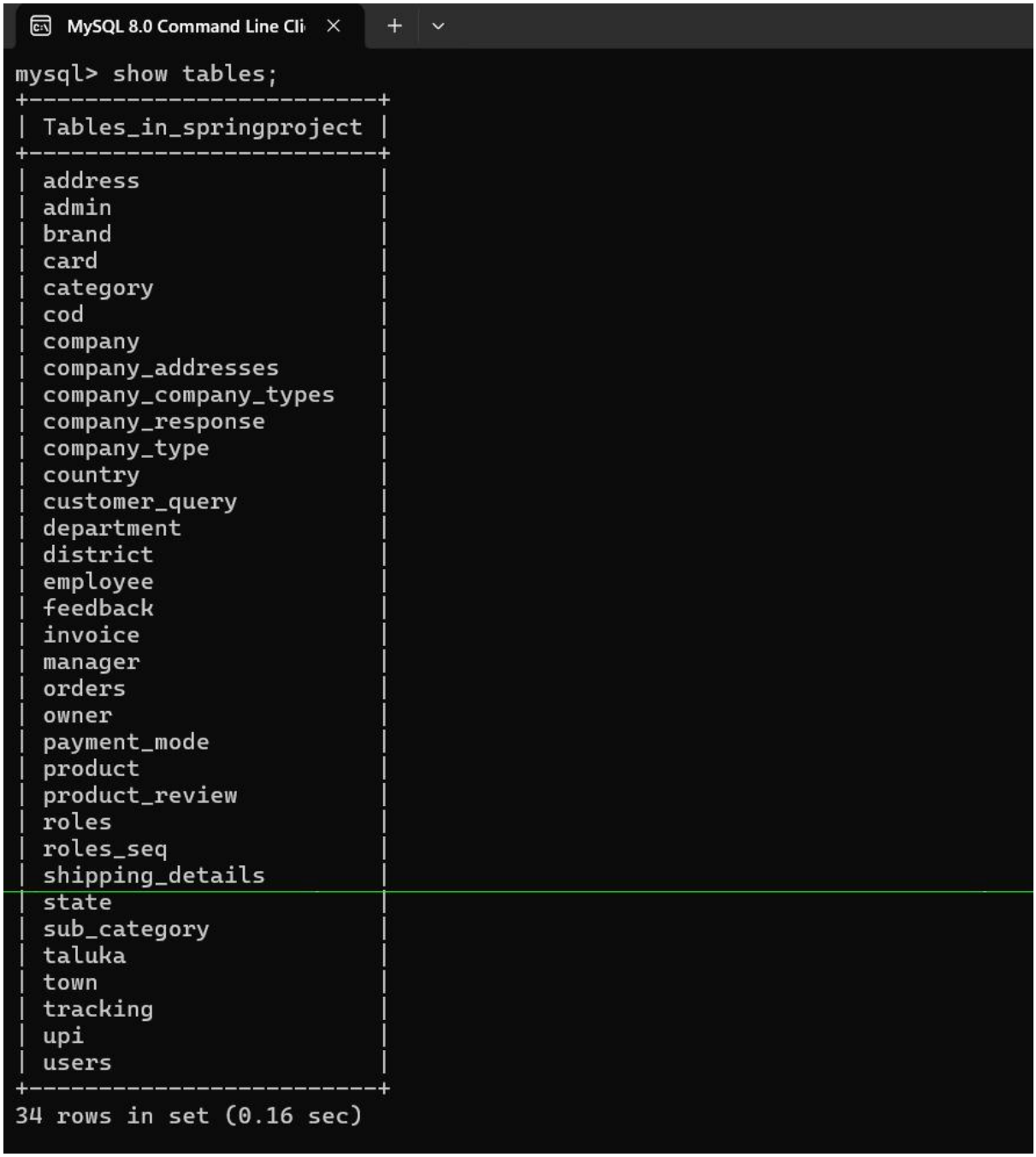
Order Management

```
Orders
  ├── Card
  ├── UPI
  ├── COD
  ├── Invoice
  ├── Tracking
  └── ShippingDetails
```

Support Management

```
CustomerQuery
       └── CompanyResponse
```

These dependencies are defined in the project design document.



---

7. Core Functionalities

User Management

- User Registration
- User Login
- Role Assignment
- Address Management

Company Management

- Company Creation
- Company Type Management
- Department Management
- Employee Management

Product Management

- Category Management
- SubCategory Management
- Brand Management
- Product Management
- Product Reviews

Order Management

- Place Orders
- Payment Processing
- Invoice Generation
- Shipping Management
- Order Tracking

Customer Support

- Feedback Submission
- Customer Queries
- Company Responses

8. Key Features

 Layered Architecture

 RESTful APIs

 MySQL Integration

 Spring Data JPA

 Hibernate ORM

 Dependency Injection

 Entity Relationships

 Modular Service Structure

 Scalable Design

 Maintainable Codebase

9. Future Enhancements

- Spring Security + JWT Authentication
- Role-Based Access Control (RBAC)
- Email Notifications
- Payment Gateway Integration
- Cloud Deployment (AWS)
- Docker Containerization
- Microservices Architecture
- API Documentation using Swagger

10. Conclusion

The E-Commerce Management System Backend is a comprehensive Spring Boot application designed using industry-standard architecture principles. The project demonstrates expertise in Java, Spring Boot, REST APIs, JPA/Hibernate, MySQL, and enterprise backend development through the implementation of 31 entities, 31 repositories, 155 service interfaces, 155 service implementations, and a complete business workflow from user registration to order tracking.